

# FlowHub – Arc42

## Architekturdokumentation

---

**CAS AI-Assisted Software Engineering (AISE)** · W4B-C-AS001 · ZH-Sa-1 · FS26 **Student:** Andreas Imboden **Version:** 2.0 (as built) · **Stand:** Juni 2026 · **Template:** arc42 v8 (adaptiert)

**Hinweis zur Version.** Version 1.0 (Februar 2026) dokumentierte die *geplante* Konzept-Architektur zu Projektbeginn. Diese Version 2.0 beschreibt die **tatsächlich gebaute** Lösung am Abgabestand und kennzeichnet bewusst, wo das Konzept (Zielbild) über das Umgesetzte hinausgeht. Die Quell-Artefakte (Projektbeschreibung, ADRs 0001–0009, Design-Dokumente) liegen im Repository: <https://github.com/freaxnx01/FlowHub-CAS-AISE>.

---

## 1. Einführung und Ziele

---

FlowHub ist ein **KI-gestützter persönlicher Eingangskorb**. Er nimmt Informationsschnipsel aus dem Alltag entgegen (ein Film-Tipp, ein Fachartikel, ein Beleg-Foto, eine Notiz), **erkennt und klassifiziert** sie automatisch und **leitet sie an den passenden Ziel-Dienst** weiter — ohne dass der Benutzer im Moment der Erfassung entscheiden muss, wohin die Information gehört.

Das adressierte Kernbedürfnis ist „**Capture without friction**“: Statt der heutigen fünf Schritte (Idee → App-Wahl → App öffnen → Kategorisieren → Ablegen) reduziert FlowHub die Erfassung auf einen einzigen Schritt. Die Klassifikation übernimmt ein **Skill-basiertes Routing-System**: ein LLM klassifiziert den Schnipsel, deterministisches Keyword-/URL-Muster-Matching dient als Fallback.

### 1.1 Aufgabenstellung

- **Funktional:** Captures über mehrere Kanäle entgegennehmen, automatisch klassifizieren, an den passenden externen Dienst routen und den Lebenszyklus jedes Captures (inkl. Fehlerfälle und Retry) nachvollziehbar machen.
- **Qualitativ:** saubere, begründete Architektur (Modular Monolith, hexagonal), dokumentierte Architekturentscheidungen (ADRs) und eine reflektierte, KI-unterstützte Entwicklungsweise.

- **Rahmen:** Umsetzung inkrementell über die fünf CAS-Blöcke (Einführung, Frontend, Service, Persistence, Deployment).

#### KI-Nutzen je Kernfunktion (ein Satz pro Funktion):

- **Erfassung:** KI macht die Ein-Schritt-Erfassung möglich — der Nutzer muss im Moment des Capture nicht entscheiden, wohin die Information gehört.
- **Klassifikation:** ein LLM erkennt Typ und Ziel-Skill des Captures, mit deterministischem Keyword-Matching als Fallback.
- **Anreicherung:** das LLM ergänzt Titel und Tags, die der Nutzer sonst manuell setzen müsste.
- **Routing:** profitiert direkt von der KI-Klassifikation — der richtige Ziel-Dienst wird ohne Nutzer-Entscheid gewählt.
- **Semantische Suche:** Embeddings erlauben Suche nach Bedeutung statt nach exakten Stichwörtern.

## 1.2 Qualitätsziele

Die Qualitätsziele sind als SMART-Anforderungen in `docs/spec/nfa.md` spezifiziert (Volltext-Tabelle in Kapitel 10). Die fünf tragenden Ziele:

Tabelle 1: Qualitätsziele (Auszug)

| Priorität | Qualitätsziel          | Konkretisierung                                                                                                            |
|-----------|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1         | <b>Performance</b>     | Capture-Listen-Abfrage (Limit $\leq 50$ ) p95 < 100 ms (NfA-01); B-Tree-Indizes auf allen häufigen Filterspalten (NfA-02). |
| 2         | <b>Betriebbarkeit</b>  | Migrations-first, idempotentes SQL als Init-Container, nie Auto-Migrate in <code>app.Run()</code> (NfA-03).                |
| 3         | <b>Zuverlässigkeit</b> | Npgsql-Connection-Pooling + transiente Retry-Policy (NfA-05); Retry + Fault-Observer in der Async-Pipeline.                |
| 4         | <b>Beobachtbarkeit</b> | Prometheus- <code>/metrics</code> -Endpoint, <code>dotnet_*</code> - und <code>http_*</code> -Serien (NfA-01).             |
| 5         | <b>Datenschutz</b>     | Verarbeitung möglichst auf eigener Infrastruktur (NfA-P1), KI-Transparenz/Provenienz (NfA-P2) — <b>noch nicht</b>          |

| Priorität | Qualitätsziel | Konkretisierung                      |
|-----------|---------------|--------------------------------------|
|           | (Ziel)        | <b>umgesetzt</b> , siehe Kapitel 11. |

## 1.3 Stakeholder

Tabelle 2: Stakeholder

| Stakeholder                     | Rolle                                                                                    | Erwartung an die Architektur                                                       |
|---------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Homelab-Operator (Primärnutzer) | Betreibt FlowHub selbst (Proxmox/Docker), nutzt bereits Vikunja, Wallabag, paperless-ngx | Reibungslose Erfassung; self-hostbar; nachvollziehbarer Verbleib jeder Information |
| „Digital Hoarders“ (Sekundär)   | Sammeln viel, ohne abzulegen                                                             | Niedrigschwellige Erfassung                                                        |
| CAS-/FFHS-Dozenten (Bewertung)  | Beurteilen die Projektarbeit                                                             | Verteilte Web-App mit KI, saubere Architektur, ADRs, KI-Einsatz-Reflexion          |

FlowHub ist bewusst ein **Single-Operator-System**, keine Multi-User-Plattform (siehe Abgrenzung, Kapitel 11 / Projektbeschreibung §10).

## 2. Randbedingungen

### 2.1 Technische Randbedingungen

| Bereich   | Festlegung                                                                         |
|-----------|------------------------------------------------------------------------------------|
| Plattform | .NET 10 / C# / ASP.NET Core (LTS)                                                  |
| Frontend  | Blazor Web App (Interactive Server) + MudBlazor als einzige Komponenten-Bibliothek |

| Bereich        | Festlegung                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------|
| API            | ASP.NET Core Minimal APIs, versioniert unter <code>/api/v1</code> , Fehler als RFC 9457 ProblemDetails              |
| Persistenz     | EF Core 10 + PostgreSQL 17 (Image <code>pgvector/pgvector:pg17</code> ); Code-First-Migrations                      |
| Messaging      | MassTransit (In-Memory in Dev/Test, RabbitMQ in Prod via <code>Bus__Transport</code> )                              |
| KI-Abstraktion | Microsoft.Extensions.AI ( <code>IChatClient</code> ); Provider via <code>Ai__Provider</code> wechselbar             |
| Build/Betrieb  | Multi-Stage-Dockerfile ( <code>sdk:10.0-alpine</code> → <code>aspnet:10.0-alpine</code> , non-root), Docker Compose |
| Konventionen   | Warnings-as-Errors, Nullable Reference Types, SemVer, Conventional Commits, 12-Factor                               |

## 2.2 Organisatorische Randbedingungen

- **CAS-Struktur:** fünf FFHS-Blöcke (1 Konzept, 2 Frontend, 3 Service/REST, 4 Persistence, 5 Deployment), Abgabe Juli 2026. Jeder Block schliesst mit einer dokumentierten Nachbereitung ab, die gegen die Moodle-Bewertungskriterien selbst geprüft wird.
- **Dokumentation:** arc42 als Architektur-Vorlage; Architekturentscheidungen als ADRs (Kapitel 9).
- **Stack-Wahl:** Die freie Technologie-Wahl wurde in der PVA bestätigt. Wo der Kurs einen Quarkus-/Jakarta-EE-Stack als Beispiel nennt, setzt FlowHub funktionale .NET-Äquivalente ein (EF Core ↔ Hibernate/Panache, MassTransit ↔ Queue-basiertes Messaging, MEAI ↔ Spring-AI). Das Programmierkriterium der Rubrik ist seit dem Update Juni 2026 framework-neutral.

## 3. Kontextabgrenzung

### 3.1 Fachlicher Kontext

FlowHub sitzt zwischen **Eingangs-Kanälen** und **Ziel-Diensten**:

- **Eingangs-Kanäle (gebaut):** Web Quick-Capture (Eingabefeld in der Blazor-AppBar) und REST-API `POST /api/v1/captures`. Ein **Telegram-Bot** ist konzipiert, aber **nicht umgesetzt**.
- **Ziel-Dienste:** Wallabag (Read-Later), Vikunja (Tasks/Kanban) — beide als `ISkillIntegration`-Adapter **gebaut**; paperless-ngx (DMS) in der Live-Demo; Obsidian/GitLab als Wissens-Ablage **geplant**. Fällt keine Zuordnung, bleibt der Capture in der FlowHub-Inbox (PostgreSQL).

Skill → Ziel-Dienst (Konzept-Mapping): Artikel → Wallabag; Homelab/Buch/Film → Vikunja; Dokument → paperless-ngx; Wissen/Zitat → Obsidian/GitLab; generisch → Inbox. **Wired am Abgabestand:** Wallabag und Vikunja.

## 3.2 Technischer Kontext

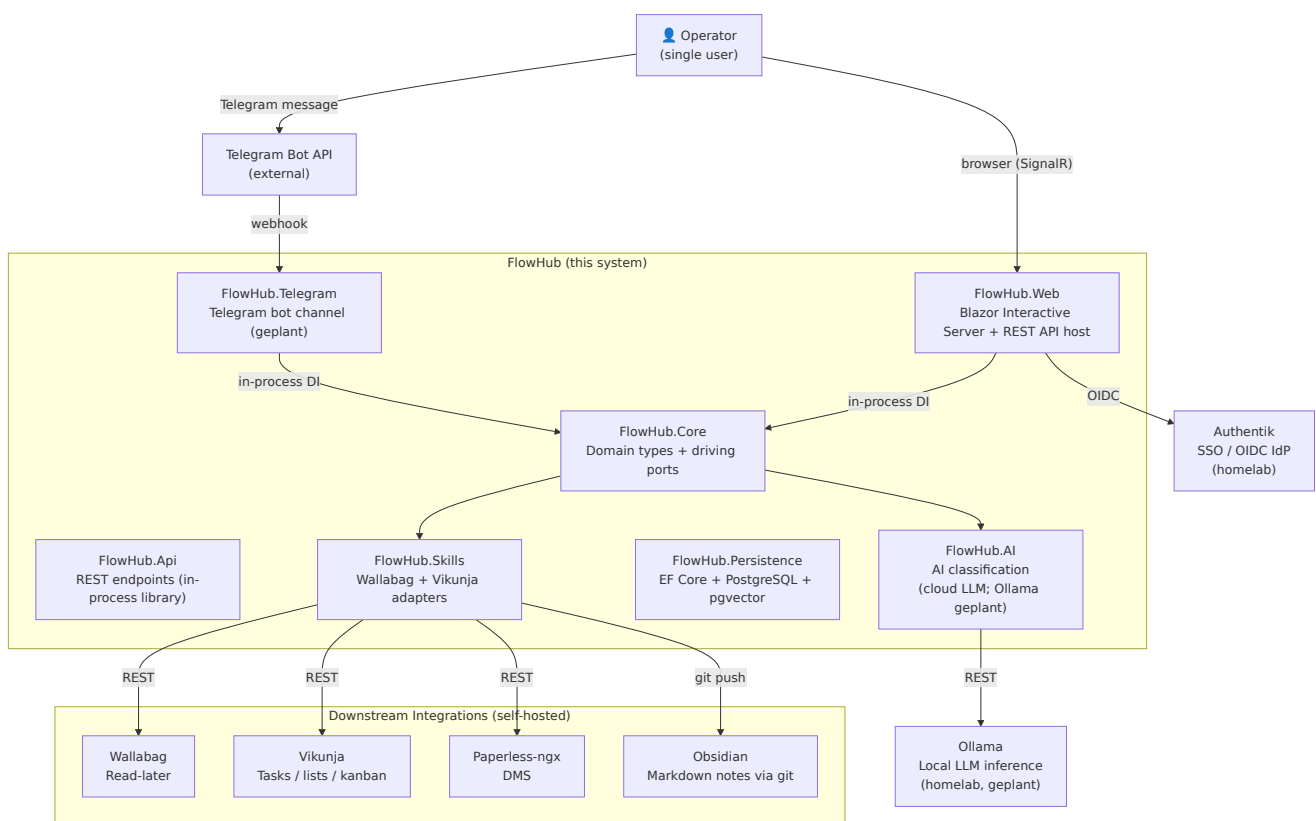


Abbildung 1: System-Kontext (C4 Level 1)

**Ist-Stand-Hinweis.** Das Kontextdiagramm zeigt das Gesamtbild inkl. geplanter Bausteine. Am Abgabestand umgesetzt sind die sechs Projekte `FlowHub.{Web,Core,Api,AI,Persistence,Skills}` mit Wallabag- und Vikunja-Adapttern; **Telegram-Kanal, lokales Ollama, Authentik/OIDC und der Obsidian-Pfad sind geplant** (KI läuft heute über Cloud-Provider, Auth über Dev-Bypass). paperless-ngx ist nur in der Live-Demo angebunden.

| Beziehung                | Mechanismus                                                               |
|--------------------------|---------------------------------------------------------------------------|
| Operator → Web-UI        | HTTP + SignalR (langlebiger Blazor-Interactive-Server-Circuit)            |
| Web / API → Core         | In-Process-DI (kein HTTP für UI-Daten, ADR 0001)                          |
| Core → KI-Provider       | REST (Cloud: OpenRouter/Mistral heute; lokales Ollama als Ziel, ADR 0007) |
| Skills → externe Dienste | HTTP REST (Wallabag, Vikunja)                                             |
| Web → Authentik (OIDC)   | <b>geplant</b> — heute Dev-Bypass ( <code>DevAuthHandler</code> )         |

## 4. Lösungsstrategie

| Problem / Treiber                               | Lösungsansatz                                                                                                                                                                                                                                  | Begründung / ADR   |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| Klare Modulgrenzen ohne Microservice-Overhead   | <b>Modularer Monolith</b> — ein deploybarer Prozess, Fähigkeiten als getrennte .NET-Projekte, modulübergreifende Aufrufe nur über Ports in <code>FlowHub.Core</code>                                                                           | ADR 0001, ADR 0002 |
| Testbarkeit, austauschbare Infrastruktur        | <b>Hexagonale Schichtung</b> je Modul — Driving-Ports ( <code>IClassifier</code> , <code>ICaptureService</code> ) und Driven-Ports ( <code>ISkillIntegration</code> , <code>Repositories</code> ) im Core, Adapter in den Fähigkeits-Projekten | ADR 0002           |
| Entkopplung Erfassung ↔ Verarbeitung, Resilienz | <b>Asynchrone Pipeline</b> über MassTransit (In-Memory in Dev, RabbitMQ in Prod); ausgehende Integrationen synchron innerhalb der Consumer                                                                                                     | ADR 0002, ADR 0003 |

| Problem / Treiber                  | Lösungsansatz                                                                                                                                                | Begründung / ADR |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Austauschbarkeit des LLM-Providers | <b>KI-Provider-Abstraktion</b> (MEAI <code>IChatClient</code> ); Anthropic + OpenRouter; <code>KeywordClassifier</code> als deterministischer Fallback-Boden | ADR 0004         |
| KI-Suche ohne separate Vektor-DB   | <b>pgvector</b> auf derselben PostgreSQL statt zusätzlicher Infrastruktur                                                                                    | ADR 0006         |

## 5. Bausteinsicht

### 5.1 Ebene 1 — Module (Ist-Architektur)

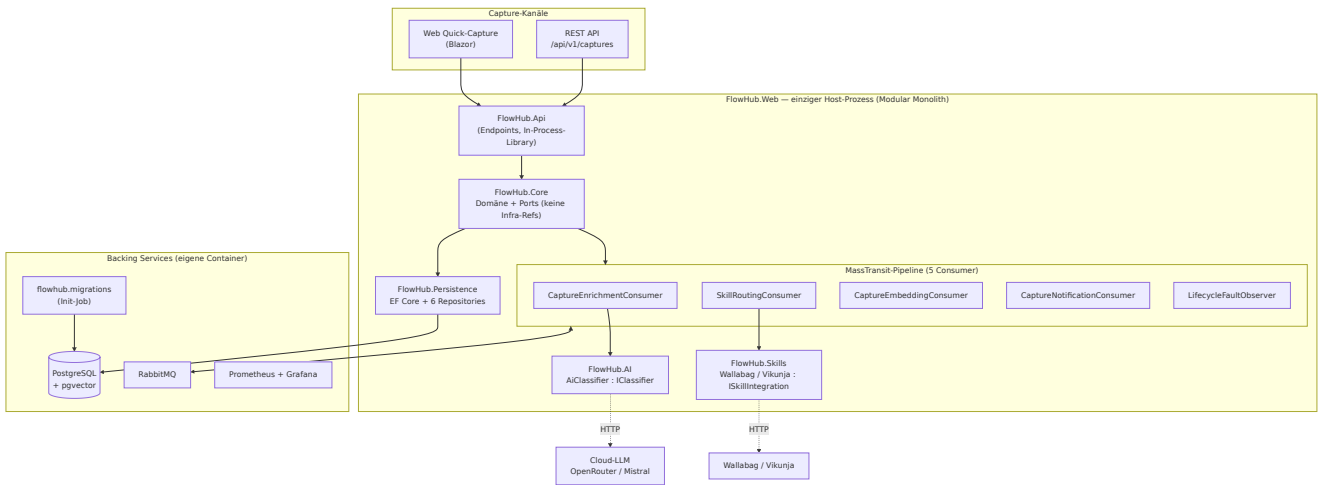


Abbildung 2: Ist-Architektur — Bausteinsicht Ebene 1 (modularer Monolith)

| Modul            | Verantwortung                                                                                                                                                                                                                                                                  |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FlowHub.<br>Core | Domänen-Typen ( <code>Capture</code> , <code>Skill</code> , <code>Health</code> ) + Driving-/Driven-Ports ( <code>IClassifier</code> , <code>IEmbeddingService</code> , <code>ISkillIntegration</code> , <code>Repository-Interfaces</code> ). Keine Infrastruktur-Referenzen. |
| FlowHub.<br>Web  | Blazor Web App (Interactive Server, MudBlazor) + MassTransit-Consumer ( <code>/Pipeline/</code> ) + Host des Minimal-API. Komponiert <code>FlowHub.Api</code> <b>in-process</b> (kein separater Container).                                                                    |

| Modul                       | Verantwortung                                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| FlowHub.<br>Api             | Minimal-API-Endpunkte ( <code>CaptureEndpoints</code> , <code>SearchEndpoints</code> , <code>AdminEndpoints</code> ) als In-Process-Library. |
| FlowHub.<br>AI              | <code>AiClassifier</code> + <code>AiEmbeddingService</code> über MEAI.                                                                       |
| FlowHub.<br>Persiste<br>nce | EF Core + PostgreSQL + pgvector; sechs <code>Ef*Repository</code> - Implementierungen.                                                       |
| FlowHub.<br>Skills          | <code>ISkillIntegration</code> -Adapter für Wallabag und Vikunja.                                                                            |

Diese sechs Projekte bilden die vollständige `FlowHub.slnx`. Telegram-Kanal und generische Integrations-Schicht sind **geplant, nicht gescaffolded**.

## 5.2 Skill-System (Kernarchitektur)

- **IClassifier** (Driving-Port): `ClassifyAsync(content) → ClassificationResult (Tags, MatchedSkill, Title?)`. Zwei Adapter: `KeywordClassifier` (Regeln) und `AiClassifier` (MEAI; füllt zusätzlich Title).
- **ISkillIntegration** (Driven-Port): `Name + HandleAsync(Capture) → SkillResult`. Auflösung per exaktem `Name == MatchedSkill`.
- **SkillRoutingConsumer** ist der Dispatcher: kein Treffer → `MarkUnhandledAsync` (Capture bleibt in der Inbox).

## 5.3 Ebene 2 — MassTransit-Pipeline (5 Consumer)

Die Async-Pipeline ist die innere Zerlegung des Host-Prozesses. Jeder Consumer ist ein eigener Baustein mit klarer Verantwortung und eigener Retry-Policy:

| Consumer                               | Reagiert auf                   | Verantwortung                                                                                        | Retry           |
|----------------------------------------|--------------------------------|------------------------------------------------------------------------------------------------------|-----------------|
| <code>CaptureEnrichmentConsumer</code> | <code>CaptureCreated</code>    | Klassifikation via <code>IClassifier</code> ; setzt <code>Classified</code> oder <code>Orphan</code> | [100ms, 500ms]  |
| <code>SkillRoutingConsumer</code>      | <code>CaptureClassified</code> | Auflösung <code>ISkillIntegration</code> per Name; Write; setzt <code>Routed / Unhandled</code>      | [500ms, 2s, 5s] |

| Consumer                    | Reagiert auf     | Verantwortung                                             | Retry           |
|-----------------------------|------------------|-----------------------------------------------------------|-----------------|
| CaptureEmbeddingConsumer    | CaptureCreated   | Best-Effort-Embedding (pgvector); parallel zum Enrichment | [500ms, 2s, 5s] |
| CaptureNotificationConsumer | Lifecycle-Events | optionale ntfy.sh-Benachrichtigung                        | —               |
| LifecycleFaultObserver      | Fault<...>       | bildet Faults auf Orphan / Unhandled ab (kein Retry)      | keine           |

Zwei Events tragen die Pipeline: `CaptureCreated` (nach dem Submit) und `CaptureClassified` (nach erfolgreicher Klassifikation). Enrichment und Embedding hängen beide am `CaptureCreated` und laufen damit parallel.

## 5.4 Ebene 2 — Skill-System (Ports & Adapter)

```

Capture submitted
→ IClassifier.ClassifyAsync(content)           (Driving-Port, FlowHub.Core)
→ ClassificationResult { MatchedSkill, Tags, Title? }
→ CaptureClassified (MassTransit-Event)
→ SkillRoutingConsumer
→ ISkillIntegration where Name == MatchedSkill (Driven-Port, FlowHub.Core)
→ integration.HandleAsync(capture) → SkillResult

```

- **IClassifier** — zwei Adapter: `KeywordClassifier` (deterministische Regeln, immer `Title = null`) und `AiClassifier` (MEAI; füllt zusätzlich `Title`, fällt bei Provider-Fehlern auf `KeywordClassifier` zurück).
- **ISkillIntegration** — ein Adapter je Ziel-Dienst, jeder kapselt HTTP/Auth/Tagging selbst. Verdrahtet: `Wallabag`, `Vikunja`. Auflösung per exaktem `Name` - Match gegen `MatchedSkill`.
- **Einen Skill ergänzen** (Erweiterungspunkt): `ISkillIntegration` in `FlowHub.Skills/<Service>/` implementieren, in den DI-Extensions registrieren, dem Classifier den neuen `MatchedSkill` beibringen. Roadmap: paperless-ngx, Wekan, Obsidian/GitLab teilen denselben Vertrag.

## 5.5 Paketstruktur

Flaches Layout `source/FlowHub.<Capability>/` (ADR 0001). Keine Sibling-Projekt-Referenzen zwischen Fähigkeiten; jede Abhängigkeit läuft über einen Port in `FlowHub.Core`.

## 5.6 Datenmodell

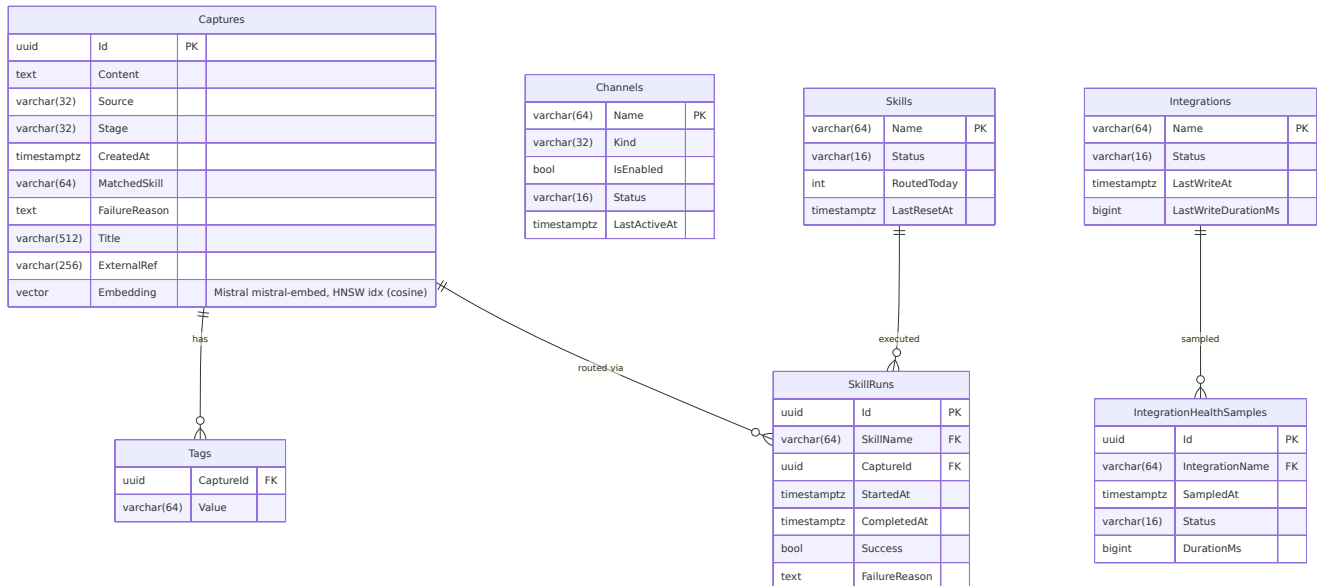


Abbildung 3: Datenmodell (Entity-Relationship)

**Captures** ist das Aggregat-Root. Das **Embedding** (pgvector-Spalte mit **HNSW**-Index — *Hierarchical Navigable Small World*, ein Approximate-Nearest-Neighbour-Index für schnelle Vektor-Ähnlichkeitssuche — mit Cosine-Distanz) kam in Block 5 hinzu; auf der öffentlichen Demo sind Embeddings deaktiviert ( `/search` → 503), siehe ADR 0006 (inkl. Amendment).

## 6. Laufzeitsicht

### 6.1 Capture-Lebenszyklus

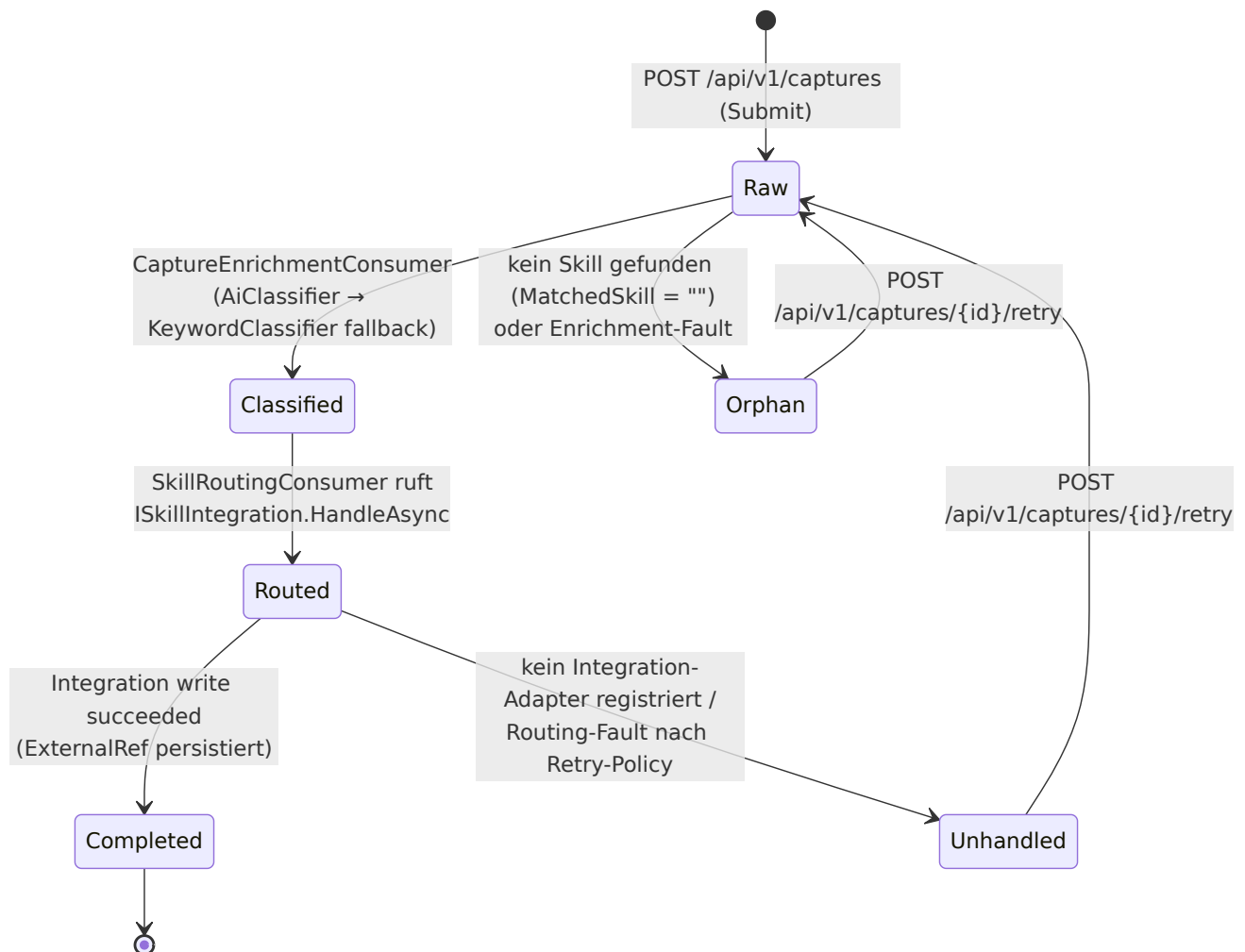


Abbildung 4: Capture-Lebenszyklus (Zustandsdiagramm)

`Raw → Classified → Routed → Completed` ist der Happy Path; `Orphan` (kein Skill / Enrichment-Fault) und `Unhandled` (kein Adapter / Routing-Fault) sind die retry-baren Terminal-Zustände — beide kehren über den Retry-Endpunkt nach `Raw` zurück.

## 6.2 Hot-Path — Submit → Skill-Write

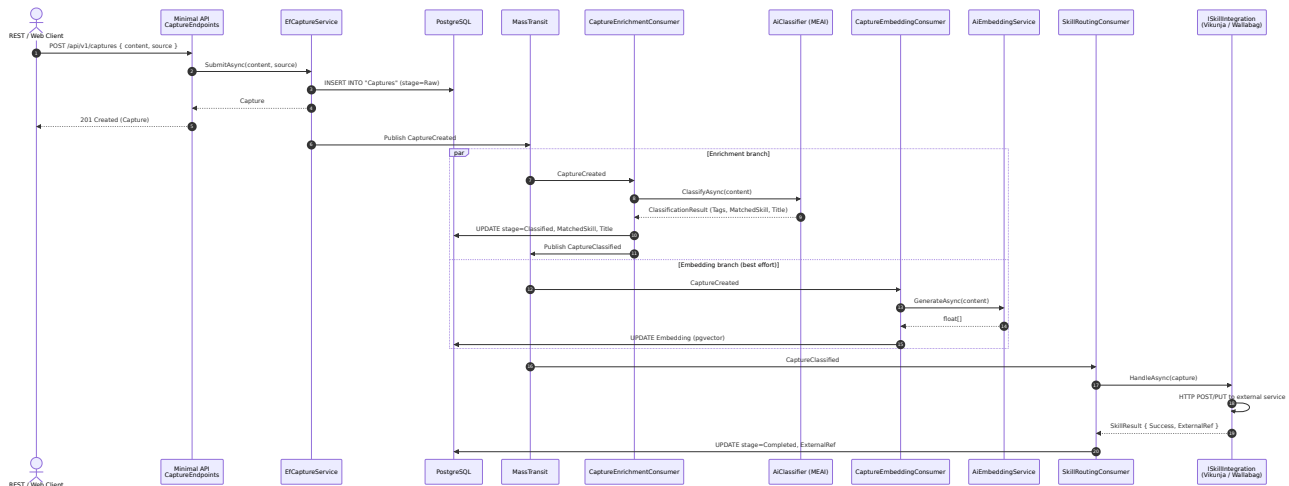


Abbildung 5: Hot-Path — Submit bis Skill-Write (Sequenz)

Wesentlich: Die HTTP-Antwort (201) erfolgt **vor** Klassifikation und Routing — der Submit-Pfad bleibt schnell, die teure Arbeit läuft asynchron. Enrichment und Embedding laufen parallel; das Embedding ist Best-Effort.

## 6.3 Enrichment — Happy Path (AI-Classifizier erfolgreich)

Gilt, wenn `Ai__Provider` und der passende API-Key gesetzt sind. `AiClassifier` macht **einen** strukturierten Provider-Call, validiert das Schema und liefert `ClassificationResult(Tags, MatchedSkill, Title)`. Ein leerer `MatchedSkill` ist ein *gültiges* Ergebnis (→ `MarkOrphanAsync`), keine Exception.

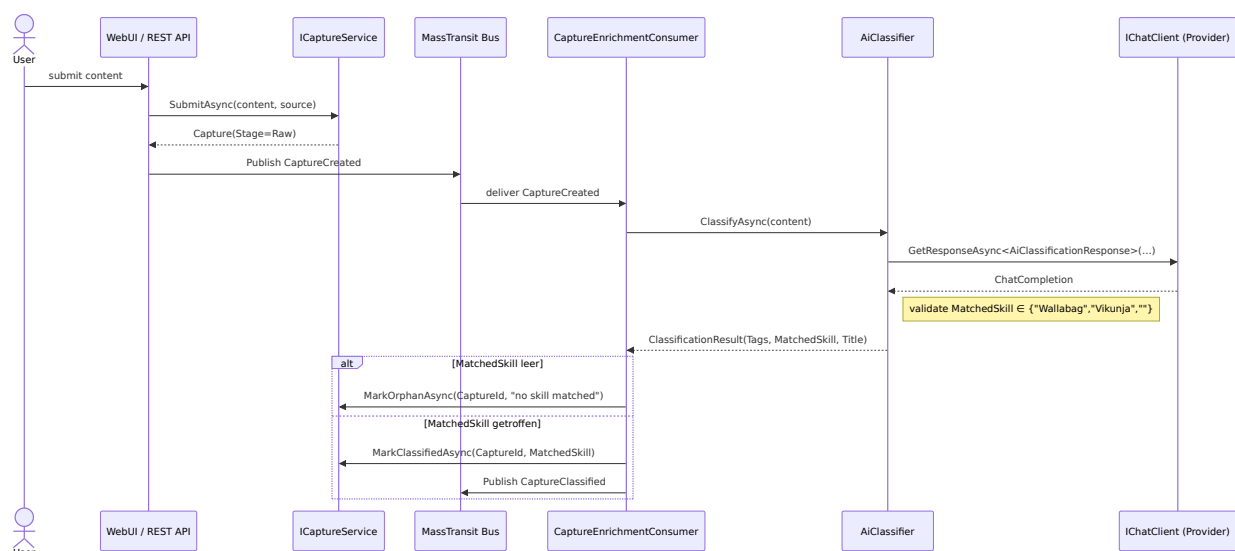


Abbildung 6: Enrichment — Happy Path (Sequenz)

## 6.4 Enrichment — Fallback (Provider-Fehler → KeywordClassifier)

Jede Exception aus `IChatClient` (Netzwerk, Timeout, JSON-Parse, Schema-Verletzung) wird **innerhalb** `AiClassifier` gefangen, auf Warning-Level geloggt (EventId 3010) und an `KeywordClassifier` als harten Boden delegiert. Der Consumer sieht in beiden Fällen ein gültiges Ergebnis — er bekommt nie eine Exception aus dem Classifier-Port. KI-Ausfall degradiert die Klassifikations-*Qualität*, nicht die *Verfügbarkeit*.

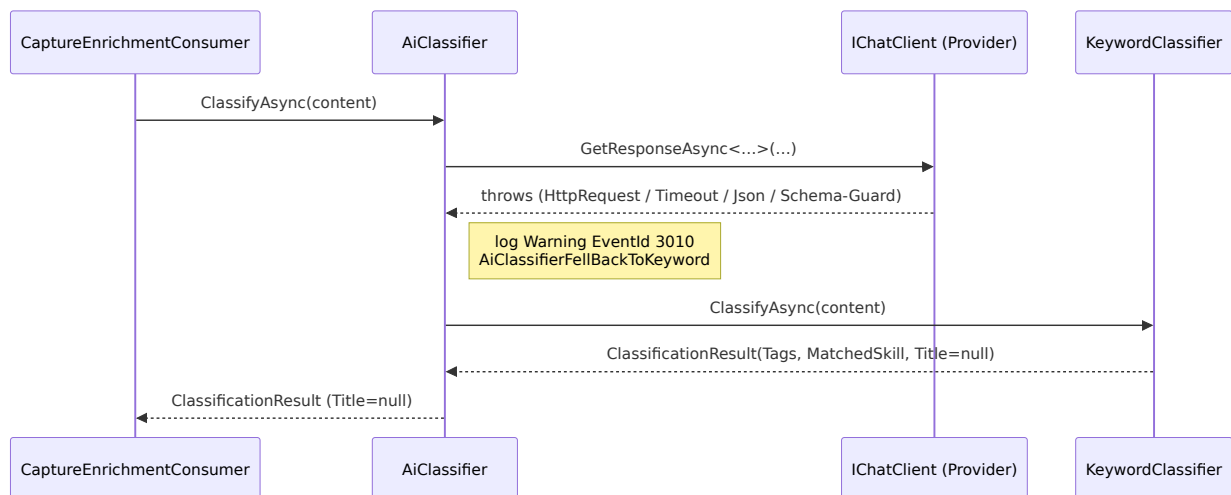


Abbildung 7: Enrichment — Fallback auf KeywordClassifier (Sequenz)

## 6.5 Skill-Routing — Erfolg

`SkillRoutingConsumer` löst die `ISkillIntegration` per exaktem `Name == MatchedSkill` auf. Kein registrierter Adapter → synchroner Terminal-Zustand `Unhandled` (kein Fault-Observer-Pfad).

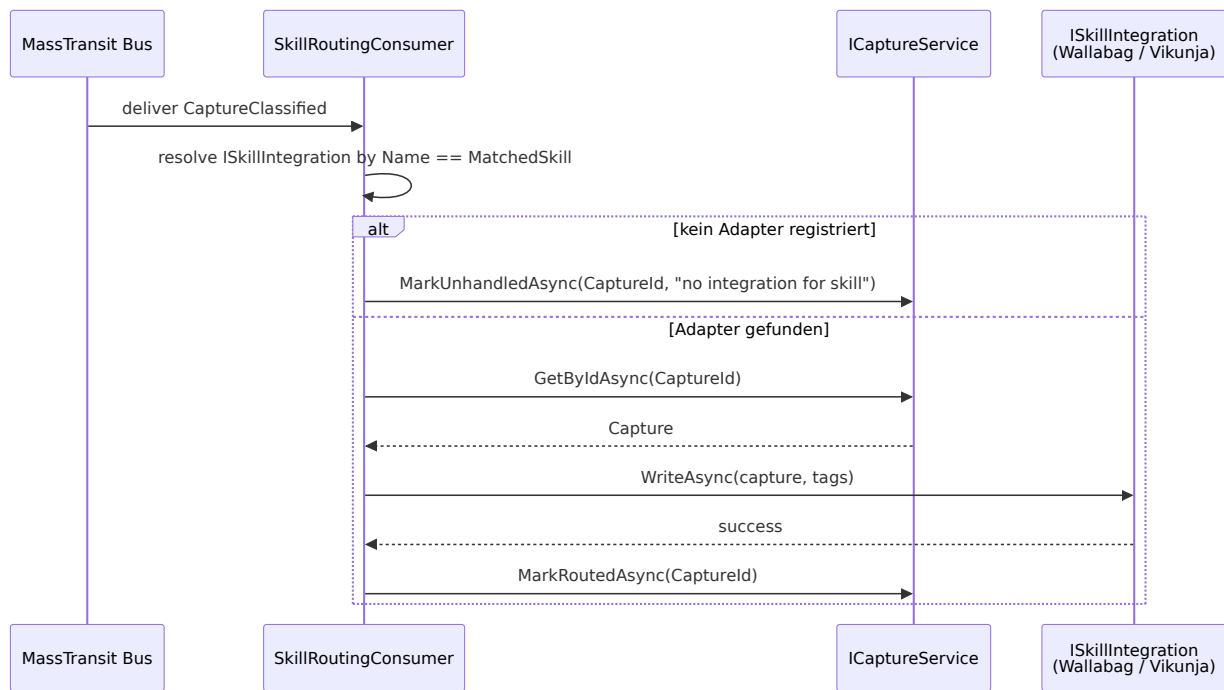


Abbildung 8: Skill-Routing — Erfolg (Sequenz)

## 6.6 Skill-Routing — Retry-Erschöpfung → Fault-Observer

Wirft `WriteAsync` bei jedem Versuch, erschöpft MassTransit das Retry-Budget ( [500ms, 2s, 5s] ) und publiziert `Fault<CaptureClassified>`. Der `LifecycleFaultObserver` (registriert **ohne** Retry-Policy, sonst Endlos-Loop) markiert den Capture `Unhandled` mit Grund aus der ersten Exception. Best-Effort: wirft `MarkUnhandledAsync` selbst, wird der Fehler geschluckt (Error, EventId 1003), um keine `Fault<Fault<T>>`-Rekursion auszulösen.

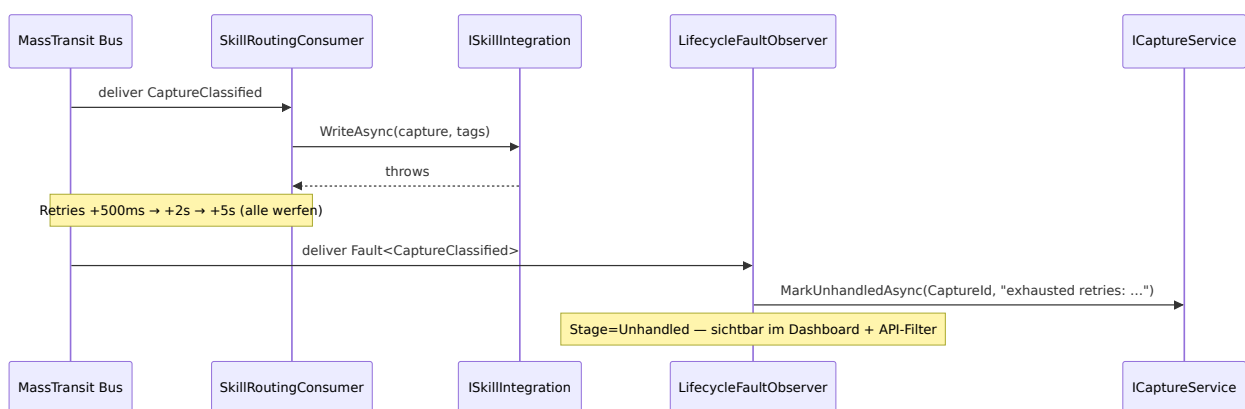


Abbildung 9: Skill-Routing — Retry-Erschöpfung & Fault-Observer (Sequenz)

Ein steckengebliebener `Unhandled`-Capture kann über `POST /api/v1/captures/{id}/retry` neu eingereicht werden (re-published `CaptureCreated`, `Lifecycle` zurück auf `Raw`).

## 6.7 Fehler- und Retry-Semantik (Zusammenfassung)

- **Per-Consumer-Retry** (ADR 0003): Enrichment [100ms, 500ms] ; Embedding + Routing [500ms, 2s, 5s] .
- **Fault-Observer:** LifecycleFaultObserver bildet jeden Fault auf den passenden Terminal-Zustand ab — Fault<CaptureCreated> → Orphan , Fault<CaptureClassified> → Unhandled (kein Retry auf den Fault selbst).
- **Embedding ist Best-Effort:** Provider-Fehler → Capture wird ohne Embedding gespeichert, die Suche degradiert auf den Nicht-Vektor-Pfad.

## 7. Verteilungssicht

### 7.1 Docker-Compose-Topologie

| Service            | Image / Build                           | Funktion                                                   |
|--------------------|-----------------------------------------|------------------------------------------------------------|
| flowhub.web        | Build aus source/FlowHub.Web/Dockerfile | Blazor-UI + Minimal-API-Host; Healthcheck /health/live     |
| flowhub.migrations | Build aus docker/migrations/Dockerfile  | Init-Job: führt efbundle vor App-Start aus (12-Factor XII) |
| postgres           | pgvector/pgvector:pg17                  | Persistenz inkl. pgvector; pg_isready -Healthcheck         |
| rabbitmq           | rabbitmq:3-management-alpine            | Message-Bus (Prod-Transport)                               |
| prometheus         | prom/prometheus:latest                  | Scrap /metrics (7 d Retention)                             |
| grafana            | grafana/grafana:latest                  | Dashboards (anonymer Viewer, provisioniert)                |

Nur flowhub.web wird beim Release gebaut und veröffentlicht; FlowHub.Api ist in den Web-Host gefaltet (ADR 0003, „as built“). Die Demo-Overlay-Compose-Datei ( demo/docker-compose.yml ) ergänzt live laufende Ziel-Dienste (Vikunja, Wallabag, paperless), einen 15-Minuten-Reset, Traefik-Labels mit Rate-Limit und eine Uptime-Kuma-Statusseite.

## 7.2 CI/CD

| Workfl<br>ow                | Trigger                                            | Schritte                                                                                            |
|-----------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>ci.yml</code>         | jeder Push / PR auf <code>main</code>              | restore → build (warnings-as-errors) → test (XPlat-Coverage) → Artefakte; gated Merge               |
| <code>release.yml</code>    | Tags <code>v*</code>                               | Image <code>flowhub.web</code> bauen, nach GHCR pushen, Release-Notes via git-cliff, GitHub-Release |
| <code>migrations.yml</code> | Push auf <code>main</code> mit Migrations-Änderung | self-contained <code>efbundle</code> als 30-Tage-Artefakt                                           |

**Automatisierungsgrenze:** Build/Test, Image-Publishing und Migrations-Bundle sind automatisiert; das **Environment-Rollout** ist bewusst manuell (ein `docker compose up` -Runbook), kein Auto-Deploy-on-Tag (ADR-Begründung in `docs/ci-cd.md` ).

## 8. Querschnittliche Konzepte

„Querschnittlich“ = übergreifend: Themen, die nicht zu einem einzelnen Baustein gehören, sondern viele zugleich betreffen. Der Begriff ist die deutsche arc42-Bezeichnung für „Crosscutting Concepts“.

### 8.1 KI-Integration (ADR 0004, 0007)

FlowHub setzt **zwei substantielle KI-Rollen** ein: **(1) LLM-gestützte Klassifikation und Anreicherung** ( `AiClassifier` — Typ-/Skill-Erkennung, Titel, Tags) und **(2) Embedding-basierte semantische Suche** (pgvector). Beide sind durch **Guardrails** abgesichert (deterministischer `KeywordClassifier` -Fallback, Schema-Validierung von `MatchedSkill` ) und durch **Human-in-the-Loop** (nicht zugeordnete oder fehlgeschlagene Captures landen in der Orphan-/Unhandled-Inbox und sind per `POST .../retry` erneut anstossbar).

Die KI ist über `Microsoft.Extensions.AI` ( `IChatClient` ) abstrahiert. Zwei Adapter sind verdrahtet — Anthropic (nativ via `Anthropic.SDK` ) und OpenRouter (via `Microsoft.Extensions.AI.OpenAI` ) — die Provider-Wahl läuft über `Ai__Provider` ,

ohne Code-Änderung. Die Klassifikation nutzt **strukturierte Ausgabe**: das Antwort-Schema wird aus einem DTO mit `AllowedValues` generiert, sodass der Provider `MatchedSkill` nur aus der erlaubten Menge liefern kann; der Code revalidiert das Ergebnis zusätzlich defensiv. Jede Provider-Exception wird innerhalb `AiClassifier` gefangen und auf `KeywordClassifier` zurückgeführt (Warning, EventId 3010) — KI-Ausfall senkt die Qualität, nicht die Verfügbarkeit. Kosten- und Latenz-Guards: `MaxOutputTokens = 300` , `Temperature = 0.2` , 10 s HTTP-Timeout.

## 8.2 Persistenz (ADR 0005, 0006)

EF Core 10 + Npgsql. Sechs Repository-Ports liegen im `FlowHub.Core` , die `Ef*Repository` -Adapter in `FlowHub.Persistence` ; `EfCaptureService` komponiert die Repositories und exponiert **nie** den `DbContext` nach aussen. Listen nutzen Keyset-/Cursor-Pagination auf `(CreatedAt DESC, Id DESC)` mit auf `[1, 200]` geklammertem Limit. Entities sind `internal sealed` (+ `InternalsVisibleTo` für Tests), sodass die Domäne nicht durch EF-Mapping-Attribute verunreinigt wird. Die semantische Suche liegt als pgvector-Spalte mit HNSW-Cosine-Index auf **derselben** PostgreSQL — keine separate Vektor-Datenbank.

## 8.3 Asynchrones Messaging (ADR 0002, 0003)

MassTransit trägt die Pipeline; die Endpoint- bzw. Queue-Namen werden via `SetKebabCaseEndpointNameFormatter` im **kebab-case** vergeben — Kleinbuchstaben mit Bindestrichen, z. B. `capture-enrichment` , `skill-routing` . Der Transport ist umschaltbar: In-Memory in Dev/Test, RabbitMQ in Prod ( `Bus__Transport` ). Jeder Consumer hat seine eigene Retry-Policy; der `LifecycleFaultObserver` ist die zentrale Fault-Senke (Kapitel 6.6).

## 8.4 Beobachtbarkeit

OpenTelemetry instrumentiert ASP.NET Core und die .NET-Runtime; der Prometheus-Exporter liefert `/metrics` ( `dotnet_*` - und `http_*` -Serien), Grafana ist provisioniert. Health-Endpunkte: `/health/live` und `/health/ready` . MEAI- ( `gen_ai.*` ) und MassTransit-Traces sowie der **OTLP**-Export (OpenTelemetry Protocol — das Wire-Format, um Traces/Metriken an einen Collector zu senden) sind **vorbereitet, im Abgabe-Build aber nicht aktiv** (ADR 0009) — siehe technische Schulden, Kapitel 11.

## 8.5 Logging-Policy (ADR 0008)

Serilog schreibt strukturiert nach stdout (12-Factor XI). Log-Aufrufe nutzen verpflichtend source-generierte `LoggerMessage` (Analyzer CA1848/CA1873). Eine **Allow-List** definiert loggbare Felder ( `CaptureId` , `Stage` , `MatchedSkill` , Dauern ...); verboten sind Capture-Body, Title, Absender-Handles, Embeddings, Prompts/Responses und Secrets. Ein `PiiScrubbingEnricher` greift als Defense-in-Depth; `ex.Message` wird nicht als Feld geloggt (nur der Exception-Typ). EventId-Namespace: 1xxx Pipeline, 2xxx Skills, 3xxx KI, 5xxx Persistenz, 9xxx Compliance.

## 8.6 Telemetry- & PII-Policy (ADR 0009)

Span-Tags sind auf eine `flowhub.*` -Allow-List mit ausschliesslich Low-Cardinality-Werten beschränkt; High-Cardinality-Grössen landen in Histogrammen/Countern statt als Tags. Ein `TagAllowListProcessor` plus `FlowHubActivitytags` -Helper setzen das im Code durch.

## 8.7 Fehlerbehandlung

Alle API-Fehler werden als RFC 9457 ProblemDetails serialisiert (stabile `type` -URLs je Fehlerklasse). FluentValidation prüft an der API-Boundary und liefert maschinenlesbare Validierungsfehler.

## 8.8 Teststrategie

TDD ist nicht verhandelbar ( `CLAUDE.md` ). Die Schichten: bUnit für Blazor-Komponenten, NSubstitute für Mocks, FluentAssertions, xUnit; **Testcontainers gegen echtes PostgreSQL** für Repository-Tests (kein In-Memory-Provider-Drift); Playwright für E2E; Live-KI-Tests sind trait-gegatet ( `[Trait("Category", "AI")]` ) und aus der Default-Suite ausgeschlossen. Abgabestand: **294 Offline-Tests grün, 0 Fehler, 0 übersprungen**. Volltext und Reconciliation-Tabelle in `docs/spec/testing-strategy.md` .

---

## 9. Architekturentscheidungen (ADR)

---

Volltext je ADR in `docs/adr/` . Alle neun ADRs haben den Status *Accepted* — d. h. die Entscheidung wurde getroffen und ist in der Lösung umgesetzt (im Gegensatz zu *Proposed*, *Rejected* oder *Superseded*).

Tabelle 3: Architekturentscheidungen (ADR-Übersicht)

| ADR  | Titel                                      | Entscheidung (Kurz)                                                                                                                                         |
|------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0001 | Frontend Render Mode & Architecture        | Blazor Interactive Server; UI ruft Services in-process (kein REST für UI); OIDC/Authentik; flaches <code>source/</code> -Layout; Web ist selbst ein Channel |
| 0002 | Service Architecture & Async Communication | Modularer Monolith (logischer, kein physischer Split); MassTransit-Bus; ausgehende Integrationen synchron in Consumern; kein gRPC; <code>/api/v1</code>     |
| 0003 | Async Pipeline (MassTransit)               | Zwei Events ( <code>CaptureCreated</code> , <code>CaptureClassified</code> ); Per-Consumer-Retry; <code>LifecycleFaultObserver</code> ; Kebab-Queue-Namen   |
| 0004 | AI Integration — Provider-Abstraktion      | MEAI <code>IChatClient</code> ; Anthropic + OpenRouter; strukturierte Ausgabe; Keyword-Fallback                                                             |
| 0005 | Persistence — EF Core + PostgreSQL         | EF Core 10, Code-First-Migrations, Cursor-Pagination, interne Entities, Migrations als Init-Container                                                       |
| 0006 | Vector Search — pgvector + Embeddings      | pgvector auf bestehender Postgres, HNSW-Cosine, Embedding off-request-path; OpenAI-kompatibler Provider via ENV                                             |
| 0007 | LLM Hosting                                | <i>Ziel:</i> lokales Ollama als Default, Cloud als expliziter Opt-in — <b>noch nicht umgesetzt, Cloud ist Live-Default</b>                                  |
| 0008 | Logging Policy                             | Kein PII/Capture-Body im Log; source-gen <code>LoggerMessage</code> ; Allow-List + Scrubber                                                                 |
| 0009 | Telemetry & PII Policy                     | OTel-Span-Tag-Allow-List ( <code>flowhub.*</code> ), Low-Cardinality; Tracing im Abgabe-Build noch nicht aktiv                                              |

Projektbeschreibung §7 listet zusätzlich die frühen Plattform-/Strategie-Entscheidungen (PE-1...PE-7), entkoppelt von den Implementierungs-ADRs.

## 10. Qualitätsanforderungen

SMART-Anforderungen aus `docs/spec/nfa.md` :

Tabelle 4: Qualitätsanforderungen (SMART-NfA)

| ID      | Kategorie       | Ziel                                                                                                       | Messung                                                                             |
|---------|-----------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Nf A-01 | Performance     | Capture-Listen-Abfrage (Limit $\leq 50$ ) p95 < 100 ms                                                     | OTel-Span-Dauer auf <code>ListAsync</code> ;<br>Testcontainers mit 10k Zeilen       |
| Nf A-02 | Performance     | B-Tree-Index auf jeder häufigen Filterspalte                                                               | <code>HasIndex</code> im Modell + Migration; <code>EXPLAIN</code> zeigt Index-Scans |
| Nf A-03 | Betriebbarkeit  | Code-First-Migrations, idempotentes SQL via Init-Container, kein Auto-Migrate                              | Migrationsdateien; <code>migrations script --idempotent</code> ; Init-Container     |
| Nf A-04 | Skalierbarkeit  | Volumen-Obergrenzen (Captures $\leq 100k$ , HealthSamples $\leq 10k$ /Integration, SkillRuns $\leq 500k$ ) | dokumentiert; Lasttest seedet 10k Zeilen                                            |
| Nf A-05 | Zuverlässigkeit | Npgsql-Pool + transiente Retries                                                                           | Data-Source-Optionen; Integrationstests                                             |
| Nf A-D1 | Deployment      | Image-Cold-Build < 5 min                                                                                   | <code>release.yml</code> -Build-Schritt $\leq 300$ s                                |
| Nf A-D2 | Deployment      | Veröffentlichtes Image < 200 MB komprimiert                                                                | GHCR-Layer-Größen                                                                   |
| Nf A-   | Deployment      | <code>/health/live</code> < 30 s nach Cold-Start                                                           | Compose-Healthcheck 10 s $\times$ 3                                                 |

| ID     | Kategorie          | Ziel                                                            | Messung                                                                        |
|--------|--------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------|
| D3     |                    |                                                                 |                                                                                |
| NfA-O1 | Beobachtbarkeit    | Prometheus <code>/metrics</code> exponiert                      | <code>curl /metrics</code> → 200, <code>dotnet_*</code> + <code>http_*</code>  |
| NfA-P1 | Datenschutz (Ziel) | Verarbeitung nur auf eigener Infrastruktur; LLM lokal (Ollama)  | <code>OutboundCallAuditTests</code> — <b>heute nicht erfüllt (Cloud aktiv)</b> |
| NfA-P2 | Datenschutz (Ziel) | KI-klassifizierte Captures sichtbar gekennzeichnet + Provenienz | Badge-Test + Migration + API-Felder — <b>nicht gebaut</b>                      |

## 11. Risiken und technische Schulden

Tabelle 5: Risiken und technische Schulden

| Punkt                                         | Art          | Status / Mitigation                                                                                                                                                                                                             |
|-----------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>EF-Outbox nicht verdrahtet</b>             | Tech. Schuld | Crash zwischen Persist und Publish kann einen Capture in <code>Raw</code> zurücklassen; Mitigation: manueller Retry-Endpoint + Dashboard-Sichtbarkeit (ADR 0003)                                                                |
| <b>OTLP-Tracing / KI-Metriken nicht aktiv</b> | Tech. Schuld | Im Abgabe-Build deaktiviert; Allow-List/Helper vorbereitet (ADR 0009)                                                                                                                                                           |
| <b>NfA-P1 / NfA-P2 offen</b>                  | Ziel offen   | Cloud-LLM + Cloud-Embeddings sind Live-Default; kein lokaler Ollama-Adapter, keine KI-Badges und keine <b>Provenienz-Spalten</b> — also keine Herkunftsfelder wie <code>ClassificationSource</code> (KI / Heuristik / Manuell), |

| Punkt                                            | Art                    | Status / Mitigation                                                                                                                                                                   |
|--------------------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                        | <code>ClassifiedAt</code> , <code>ConfidenceScore</code> , die festhalten, <i>wie</i> ein Capture klassifiziert wurde                                                                 |
| <b>Compliance-Audit-Tests geplant</b>            | Tech. Schuld           | <code>SerilogPiiAuditTests</code> , <code>TracingPiiAuditTests</code> , <code>OutboundCallAuditTests</code> als Block-5/geplant markiert                                              |
| <b>Semantische Suche auf Demo zurückge rollt</b> | Bewusste Einschränkung | Self-hosted Embedder getestet, dann entfernt (schwache Trennschärfe auf kleinem Datensatz); <code>/search</code> → 503; Pipeline + Tests bleiben als Deliverable (ADR 0006 Amendment) |
| <b>Idempotenz-Receiver fehlt</b>                 | Tech. Schuld           | RabbitMQ-At-least-once-Redelivery nicht abgesichert                                                                                                                                   |
| Scope Creep (zu viele Skills)                    | Projektrisiko          | Schlanke MVP-Liste, Future klar abgegrenzt                                                                                                                                            |
| Ollama zu langsam (kein GPU)                     | Projektrisiko          | Cloud-Fallback + Keyword-Baseline                                                                                                                                                     |
| Zeitdruck (Abgabe Juli)                          | Projektrisiko          | MVP bewusst schlank, Future-Features dokumentiert                                                                                                                                     |

## 12. Glossar

| Begriff                 | Bedeutung                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Capture</b>          | Eingehender Informationsschnipsel (URL, Text, Datei); Aggregat-Root mit Lifecycle-Stage, Source, MatchedSkill, Tags, Title, ExternalRef, Embedding                                     |
| <b>Skill</b>            | Kategorie/Handler für einen Capture-Typ; routet zu genau einem Ziel-Dienst. Ist-Mechanismus: <code>MatchedSkill</code> -String auf eine <code>ISkillIntegration.Name</code> abgebildet |
| <b>Channel</b>          | Eingangs-Quelle für Captures (Web Quick-Capture, REST-API; Telegram geplant). Die Web-UI ist selbst ein Channel (ADR 0001)                                                             |
| <b>Integration</b>      | Ausgehender Adapter zu einem Ziel-Dienst ( <code>ISkillIntegration</code> ); verdrahtet: Wallabag, Vikunja                                                                             |
| <b>Lifecycle-Stages</b> | <code>Raw</code> → <code>Classified</code> → <code>Routed</code> → <code>Completed</code> ; Fehler-Terminale <code>Orphan</code> und <code>Unhandled</code> (beide retry-bar)          |
| <b>MEAI</b>             | Microsoft.Extensions.AI — Provider-Abstraktion für LLMs in .NET                                                                                                                        |
| <b>EF Core</b>          | Entity Framework Core — .NET-ORM                                                                                                                                                       |
| <b>Blazor SSR</b>       | Server-seitiges Rendering mit Blazor                                                                                                                                                   |
| <b>Homelab</b>          | Selbst betriebene Server-Infrastruktur (Proxmox)                                                                                                                                       |

## Abkürzungen

| Kürzel      | Bedeutung                                                                   |
|-------------|-----------------------------------------------------------------------------|
| <b>MEAI</b> | Microsoft.Extensions.AI — Provider-Abstraktion für LLMs in .NET             |
| <b>ORM</b>  | Object-Relational Mapper (hier EF Core) — bildet Objekte auf DB-Tabellen ab |
| <b>DI</b>   | Dependency Injection — Abhängigkeiten werden injiziert statt selbst erzeugt |

| Kürzel            | Bedeutung                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------|
| <b>SSR / CSR</b>  | Server-Side / Client-Side Rendering                                                                    |
| <b>OIDC / SSO</b> | OpenID Connect / Single Sign-On — Authentifizierung über einen zentralen Identity-Provider (Authentik) |
| <b>pgvector</b>   | PostgreSQL-Erweiterung für Vektor-Spalten + Ähnlichkeitssuche                                          |
| <b>HNSW</b>       | Hierarchical Navigable Small World — Approximate-Nearest-Neighbour-Index für Vektorsuche               |
| <b>OTLP</b>       | OpenTelemetry Protocol — Wire-Format zum Senden von Traces/Metriken an einen Collector                 |
| <b>GHCR</b>       | GitHub Container Registry — hostet die veröffentlichten Docker-Images                                  |
| <b>RFC 9457</b>   | Standard für maschinenlesbare HTTP-Fehler („Problem Details“)                                          |
| <b>DTO</b>        | Data Transfer Object — einfaches Datenobjekt für die Übertragung über eine Grenze                      |

## Abbildungsverzeichnis

- **Abbildung 1:** System-Kontext (C4 Level 1) — Kap. 3.2
- **Abbildung 2:** Ist-Architektur — Bausteinsicht Ebene 1 (modularer Monolith) — Kap. 5.1
- **Abbildung 3:** Datenmodell (Entity-Relationship) — Kap. 5.6
- **Abbildung 4:** Capture-Lebenszyklus (Zustandsdiagramm) — Kap. 6.1
- **Abbildung 5:** Hot-Path — Submit bis Skill-Write (Sequenz) — Kap. 6.2
- **Abbildung 6:** Enrichment — Happy Path (Sequenz) — Kap. 6.3
- **Abbildung 7:** Enrichment — Fallback auf KeywordClassifier (Sequenz) — Kap. 6.4
- **Abbildung 8:** Skill-Routing — Erfolg (Sequenz) — Kap. 6.5
- **Abbildung 9:** Skill-Routing — Retry-Erschöpfung & Fault-Observer (Sequenz) — Kap. 6.6

# Tabellenverzeichnis

---

- **Tabelle 1:** Qualitätsziele (Auszug) — Kap. 1.2
- **Tabelle 2:** Stakeholder — Kap. 1.3
- **Tabelle 3:** Architekturentscheidungen (ADR-Übersicht) — Kap. 9
- **Tabelle 4:** Qualitätsanforderungen (SMART-NfA) — Kap. 10
- **Tabelle 5:** Risiken und technische Schulden — Kap. 11

# Literaturverzeichnis

---

- arc42 — Template für Architekturdokumentation, v8. <https://arc42.org>
- Simon Brown — *The C4 model for visualising software architecture*. <https://c4model.com>
- *The Twelve-Factor App*. <https://12factor.net>
- IETF RFC 9457 — *Problem Details for HTTP APIs*. <https://www.rfc-editor.org/rfc/rfc9457>
- Microsoft — *Microsoft.Extensions.AI* (MEAI) Dokumentation. <https://learn.microsoft.com/dotnet/ai/>
- Microsoft — *Entity Framework Core* Dokumentation. <https://learn.microsoft.com/ef/core/>
- MassTransit — Dokumentation. <https://masstransit.io>
- pgvector — *Open-source vector similarity search for Postgres* (HNSW-Index). <https://github.com/pgvector/pgvector>
- Malkov & Yashunin (2018) — *Efficient and robust approximate nearest neighbor search using HNSW graphs*. IEEE TPAMI.

# Anhang

---

Der vollständige Quellcode, die ADR-Volltexte (0001-0009), Spezifikations- und Design-Dokumente sowie die Test-Suite liegen im Repository und sind aus der Einreichungs-Seite ( `FlowHub_Uebersicht` ) verlinkt:

<https://github.com/freaxnx01/FlowHub-CAS-AISE>. Das Hilfsmittelverzeichnis und die Selbständigkeitserklärung sind dem separaten Dokument

`Eigenständigkeitserklärung` beigelegt.

---

*Arc42 v2.0 (as built), Juni 2026. Erstellt mit Unterstützung von Claude (Anthropic) gemäss den FFHS-Richtlinien für KI-Einsatz in Projektarbeiten.*